# SS-DID: A Secure and Scalable Web3 Decentralized Identity Utilizing Multilayer Sharding Blockchain

Yizhong Liu, *Member, IEEE*, Boyu Zhao, Zedan Zhao, Jianwei Liu, *Senior Member, IEEE*, Xun Lin, Qianhong Wu, *Member, IEEE*, and Willy Susilo, *Fellow, IEEE*

***Abstract*—Web3 is a revolutionary Internet paradigm that focusing decentralization, user empowerment, and intelligence. One of its key technologies is decentralized identity (DID), which has gained significant attention recently. However, existing DID solutions are not scalable enough to be compatible with the large-scale identity node applications required by Web3 across various fields. To overcome this challenge, we propose the first multilayer Web3 DID architecture utilizing sharding blockchain, which provides management, scalability, and compatibility. This architecture leverages leader shards and the main chain to establish trust, while regular shards manage DID-related transactions. Specific system processes and query optimizations are also given. Besides, formal security analysis and comprehensive simulation evaluations have demonstrated that the architecture can achieve all proposed security and performance goals, including low latency of down to 2 s and high throughput of up to 90 KTPS.**

***Index Terms*—Decentralized identity (DID), sharding blockchain, Web3.**

## I. Introduction

With the development of the Internet, the digital economic value created by users is increasing infinitely. However, under the traditional Internet paradigm, the centralized oligarchic platform controls users' rights and interests as content creators [1]. For example, video copyrights, recommendations, and earnings are ruled by Internet oligarchic platforms, such as TikTok and YouTube rather than Tiktokers or Youtubers. Therefore, users strongly desire a fair and decentralized Internet paradigm.

*Web3*, as the prototype of a new generation of the Internet, was initially proposed by Sir Tim Berners-Lee (2016 A.M.Turing Award Winner) in 1998 to describe the concept of the semantic web [2]. In 2014, Ethereum co-founder Gavin Wood proposed the new Web3 concept based on blockchain, smart contracts, and other technologies to achieve a real decentralized Internet [3]. After this proposal, a wave of Web3 research starts. As an emerging concept, the definition of Web3 and the description of its technical implementation were controversial at the beginning. After years of debate, the technology stack in Fig. 1 has gained comprehensive recognition [4].

In Web3 technology stack, distributed protocols are used at the foundational layer above the traditional Internet infrastructure layer, which includes computing, communication, and storage infrastructure. These protocols include consensus protocols, data distribution, transmission, etc. They form the underlying distributed network of Web3. In the extension layer, the network's functional performance optimization is further achieved, including cross-chain interaction, scalability, etc. The application layer adopts the underlying low-correlation development method to implement various distributed applications like decentralized finance (DeFi) [5], decentralized science (DeSci) [6], decentralized autonomous organizations (DAOs) [7] with public administration services, digital culture, and more. Finally, Web3 presents users with a ecosystem characterized by distributed digital identity and trusted identity-assets binding.

As the one f the most critical technology of Web3, the decentralized identity (DID) provides a decentralized trust authentication system for interactions between various applications. With the aid of distributed technology like blockchain, DID transforms the paradigm of centralized institutions controlling user digital identities [8]. It enables Web3 users to manage their identities autonomously and authenticate each other without relying on any third-party centralized identity providers. The integrity of identity data can be protected through distributed infrastructure like blockchain for verification and subsequent services. Furthermore, through DID, users do not need to create multiple identities on different centralized platforms, but can instead establish a decentralized universal digital identity system that's applicable across various platforms. This feature is user-centered and

Manuscript received 31 January 2024; revised 6 March 2024; accepted 15 March 2024. Date of publication 21 March 2024; date of current version 25 July 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2700200; in part by the National Natural Science Foundation of China under Grant U21B2021, Grant U22B2008, Grant 62202027, Grant 61972018, Grant 61932014, Grant U2241213, Grant 62172025, Grant U21A20467, Grant 61932011, and Grant 61972019; in part by the Beijing Natural Science Foundation under Grant M23016; in part by the Young Elite Scientists Sponsorship Program by China Association for Science and Technology under Grant 2022QNRC001; in part by the Open Topics of Key Laboratory of Blockchain Technology and Data Security by the Ministry of Industry and Information Technology of the People's Republic of China under Grant 20241111; and in part by the Yunnan Key Laboratory of Blockchain Application Technology Open Project under Grant 202105AG070005 and Grant YNB202206. *(Corresponding author: Jianwei Liu.)*

Yizhong Liu is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, also with the Key Laboratory of Blockchain Technology and Data Security by the Ministry of Industry and Information Technology of the People's Republic of China, Beijing 100040, China, also with the Yunnan Key Laboratory of Blockchain Application Technology, Yunnan 650233, China, and also with the State Key Laboratory of Cryptology, Beijing, 100878, China (e-mail: liuyizhong@buaa.edu.cn).

Boyu Zhao, Zedan Zhao, Jianwei Liu, and Qianhong Wu are with the School of Cyber Science and Technology, Beihang University, Beijing 100091, China (e-mail: boyuzhao@buaa.edu.cn; zhaozedan@buaa.edu.cn; liujianwei@buaa.edu.cn; qianhong.wu@buaa.edu.cn).

Xun Lin is with the School of Computer Science and Engineering, Beihang University, Beijing 100091, China (e-mail: linxun@buaa.edu.cn).

Willy Susilo is with the School of Computing and Information Technology, Institute of Cybersecurity and Cryptology, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: wsusilo@uow.edu.au).

Digital Object Identifier 10.1109/JIOT.2024.3380068

2327-4662 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

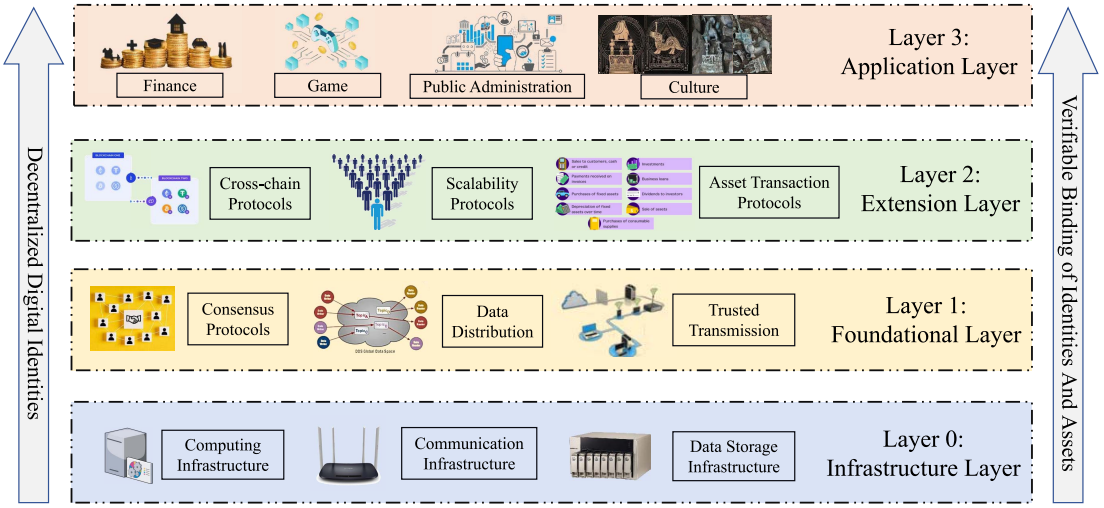



Fig. 1. Web3 technical stack.

ensures comprehensive interoperability across multiple fields and applications [9].

Many existing DID research and specific applications can be roughly divided into the following four categories [10]. Some recommended standards formulated by international standards organizations or implementation plans in some industries focus more on general DID design, including user key management and system function design. In contrast, the usage of DID in a single field focuses on optimization based on the characteristics of the field. Moreover, some DID researches make an effort on privacy protection or audit management by using cryptography algorithms.

However, DID development has faced several challenges [11], including scalability, security, privacy, regulation, and compatibility.

1) *Scalability:* As DID is designed to be self-sovereign and independent of any central authority, the challenge is ensuring the system can handle many DID-related transactions with promising performance [10]. This requires developing efficient and scalable protocols for DID creation, management, and verification.
2) *Security:* DID life cycle and usage should be secure against various attacks, including identity theft, impersonation, and data breaches. This requires implementing robust cryptographic algorithms and protocols to ensure the integrity and unforgeability of the DID-related data [12].
3) *Privacy:* DIDs are required to protect the privacy of individuals by minimizing the amount of personal data stored and shared [13]. To achieve this, privacy-enhancing technologies like zero-knowledge proofs and differential privacy are necessarily employed.
4) *Regulation:* DID aims to empower individuals with control over their own identities, which raises questions about user rights and consent. For example, individuals must have the right to access, modify, and delete their DID-related data. And DID usage should be recorded for authenticity.
5) *Compatibility:* DID ought to be compatible with a wide range of systems and platforms, including Web browsers, mobile devices, and enterprise systems [14]. This requires developing interoperable protocols and standards for DID creation, management, and verification that can be implemented across different systems and platforms.

### A. Contributions

In this article, we make the following contributions.

1) *We propose the first multilayer sharding-based secure and scalable DID architecture called SS-DID.* Identity management nodes run Byzantine fault tolerance (BFT) consensus to process DID-related transactions, ensuring decentralization, strong consistency, and low latency. Moreover, by utilizing a multilayer sharding structure, institution nodes only need to process transactions related to their shard. The main chain can realize trust between different fields.
2) *We design comprehensive and complete system processes for SS-DID.* Our system processes includes system initialization, DID life cycle, credential usage, Identity Management Institution (IMI) node management, and various query optimizations. For each part of the system process, we provide a specific and easy-to-understand scenario description. At the same time, we provide a step-by-step detailed explanation of the system process of each part.
3) *We give formal security analysis and evaluation of SS-DID.* After analysis and evaluation, our SS-DID system can achieve *unforgeability, liveness*, and *high scalability*. Especially, the most complex cross-field credential verification latency can achieve less than 2 s. Besides, our system overall throughput can achieve 90 KTPS.

### B. Related Work

*Practical Industrial Implements:* As a developing practical technology, some recommendation standards for DID have recently been proposed by standardization organizations, including the DID foundation (DIF) [15] and the DID working group of the World Wide Web Consortium (W3C) [16].

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

Also, there are many industrial implementations of DID systems, such as Microsoft-DID [21], HyperLedger Indy [22], Sovrin [23], and uPort [24]. However, these standards or implementations are only constructed for DID applications in a specific industry and are strenuous to be compatible with privacy protection methods.

*Privacy-Protected DID Schemes:* Besides, some privacy-protected DID schemes are proposed, such as CanDID [17] and Oblivira [25]. With the utilization of oracle and a combination of existing Web authentication services infrastructures, CanDID makes an effort to realize user-friendliness with strong privacy protection and solve the bootstrapping problem of the credential-issuance ecosystem. Oblivira focuses on users' privacy protection during DID resolving. In this work, based on side-channel security analysis of the W3C DID standard and universal resolver implementations, a novel TEE-based DID resolver is proposed. However, these solutions can not combined with distributed trust infrastructure like blockchain and lead to incompatibility with Web3 applications.

*Auditable and Regulatory DID Schemes:* Some works take the DID audit and regulation into consideration. For example, Hades [26] and DIDAPPER [18] consider regulation issues under pseudonyms and promote the application/interoperability of DID in on-chain decentralized applications (DApps). In Hades, full accountability and fine-grained sybil-resistance can be accomplished by smart contracts with identity-based access control. Meanwhile, DIDAPPER [18] utilizes group managers in group signatures to force open identities for auditable anonymous credentials. Yet they neglect the DID maintenance institutions' dynamic and management.

*Single Field DID Usage:* In addition to the above research on security issues of general DID, much work is also studying the practical matters of DID application in various scenarios. Typical cases include the persistent identifier layer for data catalog vocabulary (DCAT) design, which leverages DID to optimize the data set organization [27], and the DID access management framework for IoT, combining blockchain-based DID and VC [19], [20]. However, there is still a lack of research on the unified DID architecture in Web3.

*Comparison:* In general, most of these works focus on privacy, compatibility, accountability, etc., while few works consider the application of DID in Web3 and take into account the demand for scalability and cross-field management in Web3 application scenarios. Moreover, almost none of them optimize the key underlying blockchain architecture. For the sake of it, our work solves the problem for the first time by utilizing multilayer sharding architecture. We give a comparison between SS-DID and other DID in Table I.

TABLE I
COMPARISON BETWEEN SEVERAL DID SCHEMES

| DID Scheme | Regulation | Compatibility | Scalability | Web3-Applicability |
|---|---|---|---|---|
| **DIF DID** [15] | ✗ | ✓ | ✗ | ✗ |
| **W3C DID** [16] | ✗ | ✓ | ✗ | ✗ |
| **Hades** [17] | ✓ | ✗ | ✗ | ✗ |
| **DIDDAPPER** [18] | ✓ | ✗ | ✗ | ✗ |
| **DIAM-IOT** [19] | ✓ | ✗ | ✗ | ✗ |
| **BDIM** [20] | ✓ | ✗ | ✗ | ✗ |
| **SS-DID** (This Work) | ✓ | ✓ | ✓ | ✓ |

The symbol "✗" represents that the scheme does not satisfy the property, while "✓" indicates that the scheme satisfies the property.

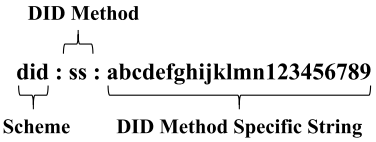



Fig. 2. W3C-DID format.

## II. PRELIMINARIES

This section gives preliminaries, including the W3C DID recommendation standard, blockchain, sharding technology, and Merkle tree.

### A. DID Recommendation Standard

The W3C Decentralized Identifiers Working Group have released the first official recommendation standard called DIDs v1.0 [16]. The DID format is regulated as a URL scheme as shown in Fig. 2. The DID string contains three fields, including DID scheme, DID method, and DID string. DID scheme is a fixed field "did." The DID method which is recorded on the blockchain mandatorily defines how to generate, parse, verify, and revoke DID-related documents. The DID method specific string is the unique identifier string based on the DID method. Thus, the entire DID identifier is globally unique.

DID document is the parsing of DID and shows how to use a specific DID. There is a one-to-one correspondence between DID and DID documents. The DID document includes the unique DID identifier, detailed information on the public key, and descriptions of other attributes of the DID holder.

### B. Blockchain and Consensus

Blockchain, a revolutionary technology that underlies Bitcoin, was first introduced by Satoshi Nakamoto in 2008 [28]. It is a decentralized database or ledger maintained by a network of participating nodes through a specific consensus algorithm. By combining several technologies and arranging blocks in a chain-like structure, blockchain enables unique features, such as decentralization, tamper-proofing, transparency, traceability, and high reliability. Our scheme utilizes BFT algorithms, such as PBFT [29], BFT-SMART [30], Tendermint [31], and Hotstuff [32] as the blockchain consensus, which allows us to achieve *strong consistency*, *liveness*, *high throughput*, and *low processing time*.

### C. Sharding Technology

The concept of sharding technology originated within the database field. All nodes are divided into multiple shards to enhance the network's scalability. In contrast to all nodes working on the whole database, each shard is responsible for maintaining its corresponding data, and processing capabilities can be increased by adding more shards as the number of nodes within the network grows. When combined with blockchain, sharding blockchain was first conceived by ELASTICO [33] to improve transaction throughput, as Fig. 3 shows. Recently, numerous studies have been conducted on sharding blockchain frameworks and its utilization in IoT

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

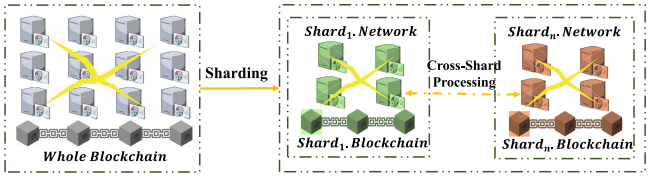



Fig. 3. Sharding blockchain technology.

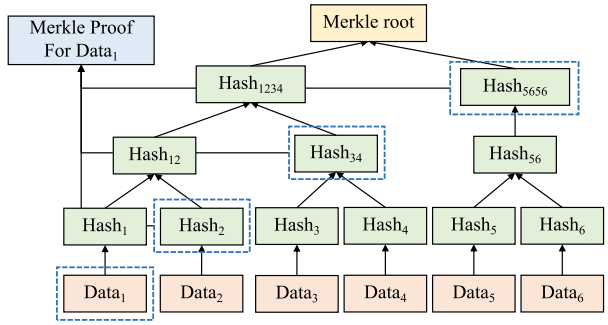



Fig. 4. Structure of a Merkle tree.

**Algorithm 1** Merkle Proof Generation

**Input:** A binary Merkle tree $MT$, Target message $m$
**Output:** A Merkle proof $M_p$

Hash value $v = Hash(m)$ and $M_p$ is empty
**for** each non-root layer in Merkle Tree **do**
  Add v' in the same layer and kid tree as v into $M_p$
  v=v'
**end for**
**return** $M_p$

or wireless communication [34], including Omniledger [35], Rapidchain [36], SSHC [37], FS [38], and Cherubim [39].

### D. Merkle Tree

A Merkle tree [40] is a tree-like data storage structure used to efficiently verify the integrity of large data sets. Each leaf node's value is the data hash, while the values of nonleaf nodes are obtained by hashing the values of their child nodes. A Merkle tree is constructed layer by layer from bottom to top until there is only one root node left, known as the Merkle root, which can represent all the data in the tree. The utilization of a collision-resistant and one-way hash function in the construction of a Merkle tree ensures that any alteration to the data will yield a distinct Merkle root. A typical binary Merkle tree is shown in Fig. 4.

In practice, to verify a single data part in the Merkle tree, using all other data parts for reconstructing the whole Merkle tree is not necessary. For example, to verify $\text{Data}_1$ on the Merkle tree in Fig. 4, we can use only $\text{Hash}_2 \| \text{Hash}_{34} \| \text{Hash}_{5656}$ as the Merkle proof. The value $\text{Hash}(\text{Hash}(\text{Data}_1), \text{Hash}_2)$ equals $\text{Hash}_{12}$, and with $\text{Hash}_{34}$, $\text{Hash}_{1234}$ can be generated. The Merkle root can be reconstructed. Detailed proof generation and verification are shown in Algorithms 1 and 2.

**Algorithm 2** Merkle Proof Verification

**Input:** Merkle proof $M_p$, target message $m$, Merkle root $M_r$
**Output:** The verification result $b$

Hash value $v = Hash(m)$
**for** each hash value $v' \in M_p$ **do**
  Update $v = Hash(v, v')$
**end for**
**if** $v = M_r$ **then**
  **return** True
**else**
  **return** False
**end if**

## III. System Overview

In this section, we describe system models, system architecture, and system goals.

### A. System Model

*Adversary Model:* A probabilistic polynomial time (P.P.T.) adversary $\mathcal{A}$ satisfies the following limits.

1) $\mathcal{A}$ cannot forge digital signatures of any honest nodes.
2) $\mathcal{A}$ is a Byzantine adversary with the ability to corrupt honest nodes within the committee. The number of corrupted nodes, denoted as $f$, satisfies $3f+1 \leq n$, where $n$ represents the total number of ATC nodes.

*Communication Model:* An authenticated channel is adopted by each two IMI nodes. The network is partially synchronous, where messages between honest nodes are sure to arrive at each other after an unknown but limited time bound $\Delta$.

### B. System Architecture

Fig. 5 illustrates the architecture of our proposed SS-DID system. In our framework, there are different scenes operating in the Web3 environment such as DeFi, decentralized healthcare, and decentralized IoT. In each scene, there are multiple institutions which can issue the credentials for users or devices, e.g., the hospitals in healthcare, the manufacturers of IoT devices, insurance companies in finance, and police offices in public administration. Compared with traditional DID systems, users can use their credentials across different scenarios seamlessly, safely, and efficiently. In our SS-DID system, there are four layers, and we will introduce the system from bottom to top layer.

*SS-DID User Layer:* This layer contains *SS-DID common user (CU)* nodes which represent the Web3 user entity. The CUs play the same roles as the users in traditional DID standards. In reality, CUs always interact with management institutions for registration and DID management. For instance, when two CUs want to trust each other, they must register in the SS-DID system. Afterward, two CUs can verify credentials from the IMIs and achieve point-to-point trust.

*SS-DID Field User Management Layer:* This layer contains *SS-DID IMI* nodes which take responsibility for CUs' DID-related transactions processing on the blockchain network. The transactions include DID life cycle management and credential usage. In practice, IMIs in different industry

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

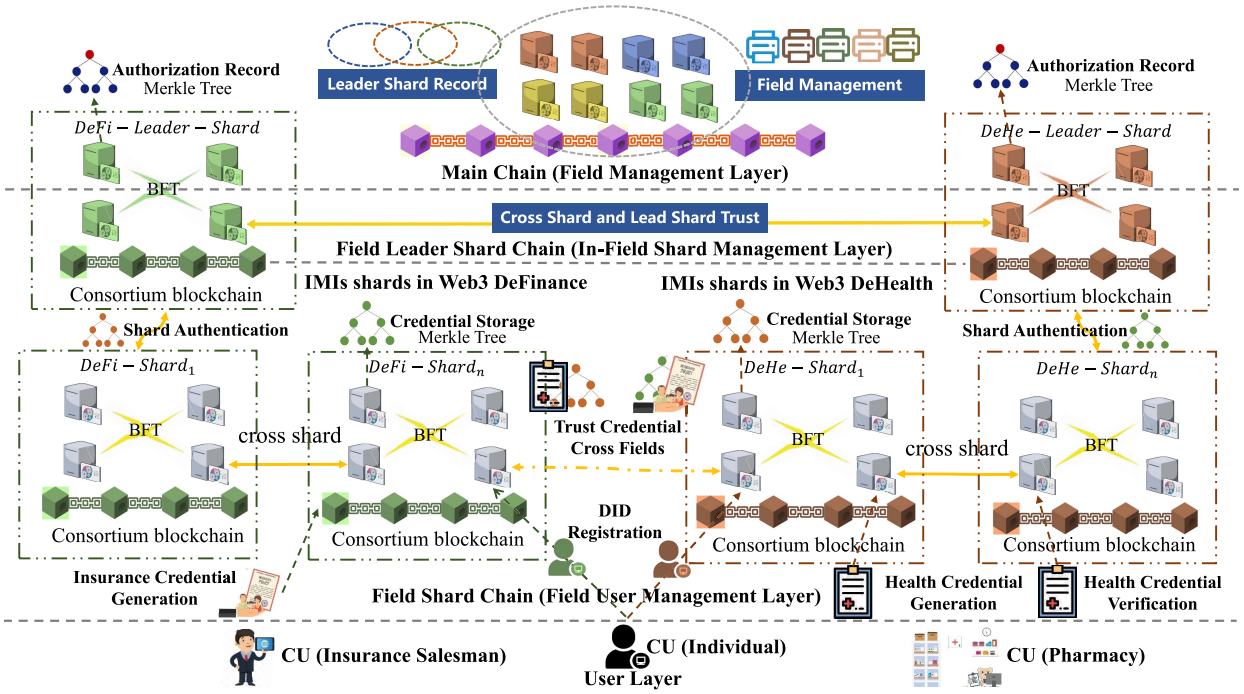



Fig. 5. Overview of our proposed secure and scalable DID system.

fields denoted as $\mathsf{F}^1, \ldots, \mathsf{F}^m$ always have different DID and credentials management requests. In each field, IMIs are divided into different shards denoted as $\mathsf{S}^1, \ldots, \mathsf{S}^u$. Each shard contains $n = 3f + 1$ IMI nodes that run BFT algorithm to maintain a consortium blockchain $\mathsf{C}$ where DID lists, DID documents, and credentials usage are recorded.

*SS-DID In-Field Shard Management Layer:* This layer comprises different field leader shards $\mathsf{F}^i.\mathsf{S}^L$ and establishes trust between them. The primary function of each field leader shard $\mathsf{S}^L$ is to manage shard changes within the field. At the same time, the field leader shard can regularly store the information processed by each shard in the field on $\mathsf{S}^L.\mathsf{C}$ in the form of a Merkle tree, which is equivalent to authorizing the cross-field use of this information.

*SS-DID Field Management Layer:* In this layer, nodes from different fields jointly maintain the main chain of the system, which records the updated records of shard leaders in different fields and the management process of each shard in the field, which is recorded in the form of a Merkle tree.

More system process details are illustrated in Section IV.

### C. System Goal

We aim to design a secure and scalable DID system that satisfies the following properties.

1) *Security:* Security contains the following meanings.
   a) *Unforgeability of DID:* Any DID-related message cannot be forged by other entities. (Unforgeability of DID implies consistency among IMI nodes, which means honest IMI nodes must have an identical view of consensus results of DID-related transactions.)
   b) *System Liveness:* The valid request submitted to the SS-DID system must be processed, and information must be written on the sharding blockchain after a specific time.
2) *Scalability:* Scalability is defined from two aspects.
   a) *System Scalability:* The entire system's ability to handle DID-related can grow as the IMI nodes increase.
   b) *Low Latency:* The authentication latency can be controlled in ten seconds or seconds for specific scenarios.
   c) *High Throughput:* The system can handle massive execution requests and reaches ten thousands or even hundreds of thousands in throughput.

## IV. Secure and Scalable DID System

Next, we give more detailed system flows of SS-DID. We divide the system processes into different parts, including system initialization, DID life cycle, credential usage, and IMI shard management.

### A. System Initialization

System multilayered initialization is performed as follows.

*Step 1:* In each shard $\mathsf{S}$, $\mathsf{S.IMI}$ generates a key pair $(apk, ask)$ used in $\mathsf{BFT}$ and initializes a consortium blockchain. The key pair $(apk, ask)$ will be used to generate the aggregate signatures as the commitment certificate. After this step, field user management layer initialization has finished.

*Step 2:* In each field $\mathsf{F}$, all shards run a selection process to select the field leader shard $\mathsf{F}.\mathsf{S}^L$. After this step, field leader layer initialization has finished.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

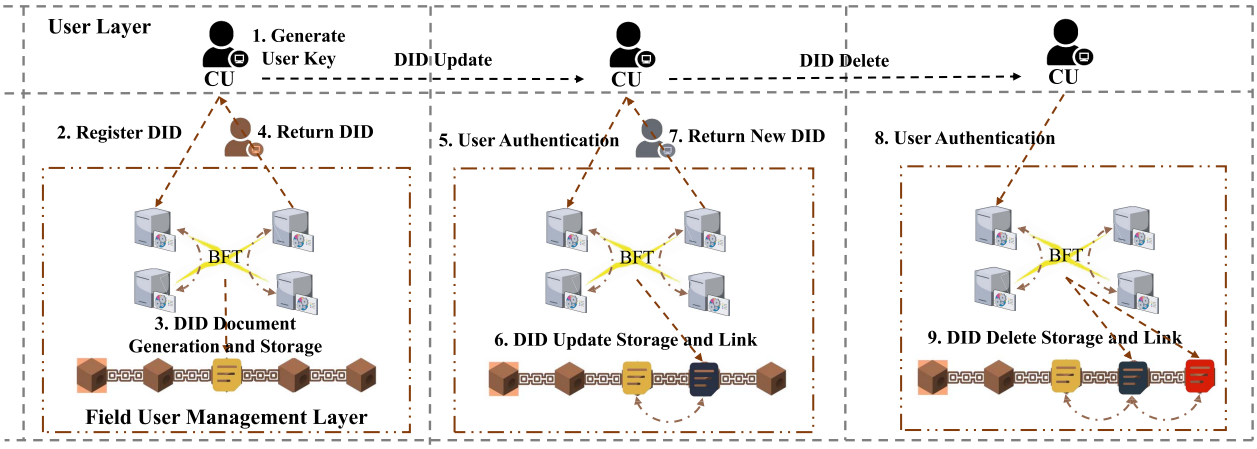



Fig. 6. Life cycle of DID.

*Step 3:* In each field leader shard $\mathsf{F.S}^L$, select some IMI nodes randomly.

*Step 4:* IMI nodes from different leader shards $\mathsf{F.S}^L$.C jointly run the alliance chain as the main chain MC of the entire system and record the information of their respective leader shards on the MC. After this step, the top layer initialization is finished.

### B. DID Life Cycle

As depicted in Fig. 6, the complete DID life cycle contains DID registration, update, delete, and optional authorization list management. Thus, we will illustrate the specific processes part by part.

*1) DID Registration:* The following steps need to be executed only once in the whole life cycle to complete DID registration.

*Step 1:* In DID, the user's key is self-generated and self-managed. The CU should first generate a key pair $(upk, usk)$, including the user public key $upk$ and user private key $usk$.

*Step 2:* After key generation, the CU initiates a registration request $TX_{\text{Reg}} = \langle \text{'}DIDReg\text{'}, upk, mes, T\rangle_{usk}$ to the $\mathsf{S}^1$.IMIs. The timestamp $T$ is used to maintain request freshness. $\langle\rangle_{usk}$ means the message is signed by the users' private key $usk$.

*Step 3:* $\mathsf{S}^1$.IMIs run BFT to verify the $TX_{\text{Reg}}$, generate the DID document like the process in Section II-A, and record the DID document on the C. Then, the proof of DID document $\langle\text{DID-Doc}\rangle_{\sigma^1}$ is sent to CU. $\langle\rangle_{\sigma^i}$ means the message is multisigned by the $\mathsf{S}^i$.IMIs private key $ask$s after BFT operation and can be verified by $\mathsf{S}^i$.IMIs' public key $apk$s.

*Step 4:* After receiving the proof of DID document $\langle\text{DID-Doc}\rangle_{\sigma^1}$ from $\mathsf{S}^1$.IMIs, the CU verifies the DID document and extracts the DID from the DID document.

*2) DID Update:* After registration, the CU can change the key or expiration date of DID document. The following steps need to be executed to update DID.

*Step 5:* The CU sends an update request $TX_{\text{Upd}} = \langle \text{'}DIDUpate\text{'}, Mes_{\text{new}}, DID, T\rangle_{usk}$.

*Step 6:* $\mathsf{S}^1$.IMIs run BFT to verify the $TX_{\text{Upd}}$, update the DID document, and record modification on the C. Then, the proof of DID document modification $\langle\text{DID-Doc-Up}\rangle_{\sigma^1}$ is sent to CU. The modification record can be linked to the block number where the last modification record or the initial DID document is located, enabling traceability from the DID modification.

*Step 7:* After receiving the proof of DID document update $\langle\text{DID-Doc-Up}\rangle_{\sigma^1}$ from $\mathsf{S}^1$.IMIs, the CU verifies the DID document and extracts new DID from the DID document.

*3) DID Deletion:* The CU can revoke the old DID to end the DID life. The following steps need to be executed.

*Step 8:* The CU sends a deletion request $TX_{\text{Del}} = \langle \text{'}DIDDel\text{'}, DID, T\rangle_{usk}$.

*Step 9:* $\mathsf{S}^1$.IMIs run BFT to verify the $TX_{\text{Del}}$ and record modification on the C. Then, the proof of DID document deletion $\langle\text{DID-Del}\rangle_{\sigma^1}$ is sent to CU. The deletion record can be linked to the initial DID document is located. After deletion, all the update requests of this DID will be considered as invalid operations.

*Step 10:* After receiving the deletion proof of DID document $\langle\text{DID-Doc-Del}\rangle_{\sigma^1}$ from $\mathsf{S}^1$.IMIs, the CU will delete the local DID document.

*4) Optional DID Authorization:* The $\mathsf{CU}_0$ may authorize other agent CUs to use DID. For example, in an enterprise, superior members can authorize DID and keys to lower-level members for work convenience. To realize simple DID authorization, $\mathsf{CU}_0$ should send an update request $TX_{\text{Upd}}$ which includes the delegated DID to $\mathsf{S}^i$.IMIs.

After receiving the $TX_{\text{Upd}}$, $\mathsf{S}^i$.IMIs run BFT to update the DID delegation message on C and send the proof $\langle\text{DID-Doc}\rangle_{\sigma^i}$ to $\mathsf{CU}_0$. From Section IV-C, the credential usage of $\mathsf{CU}_0$'s DID can be recorded on the chain. Therefore, the operation of agent DID is also completely traceable and guaranteed to be trustworthy. Besides, the system supports more flexible authorization operations due to the openness of the architecture.

*5) DID Query Optimization:* The system running time can be divided into multiple epochs. At the beginning of each

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

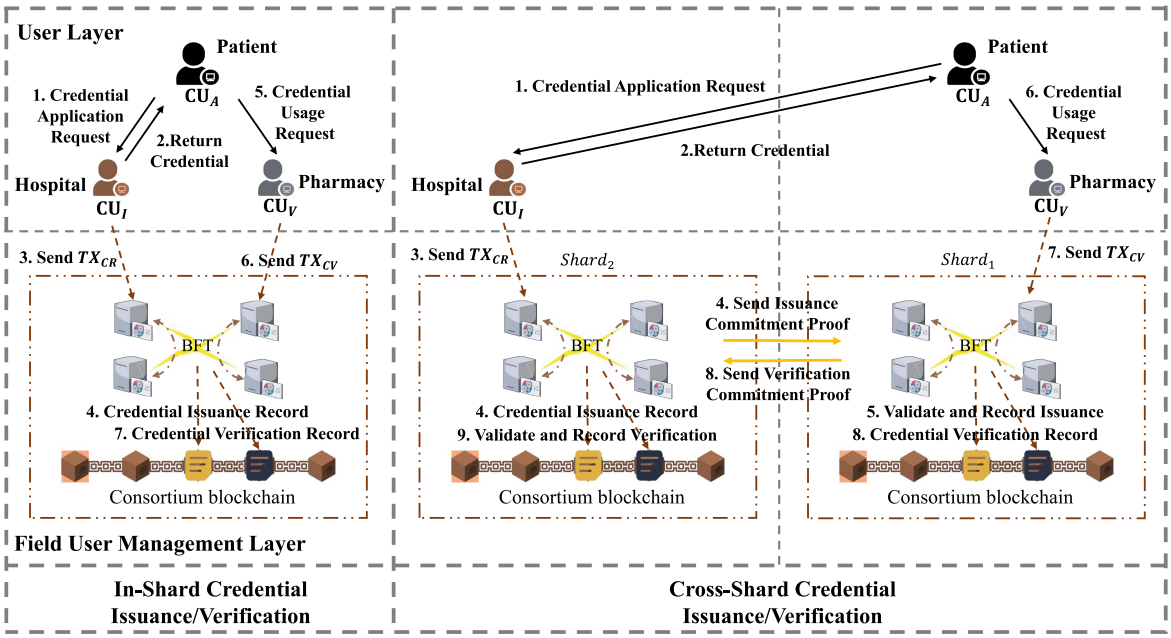



Fig. 7. In-field usage of credential.

epoch $e_k$, each shard $\mathsf{S}^i$.IMIs run BFT to update the DID states list $DSL_k$ at $e_k$ from $DSL_{k-1}$ at $e_{k-1}$ and the chain growth $\mathsf{C}^{k-1}$ in $e_{k-1}$.

Through query optimization, there is no need to search from the beginning of the entire chain when querying the DID status in $e_k$. Instead, the latest DID state can be obtained by simply looking up the chain growth in $e_k$ and the $DSL_k$.

### C. Credential Usage

Once the DID life cycle design is complete and implemented, the organization is required to issue and verify credentials based on the user information during the actual use of DID. In our system, the DID of the credential requester and issuer is registered in one field (possibly on different shards), while the DID of the verifier may belong to another field.

Therefore, based on the relationship between the credential applier, the issuer, and the verifier, we divide the usage process of credentials into different parts: in-shard credential issuance/verification, cross-shard credential issuance/verification, and cross-field credential verification. At the same time, in order to facilitate understanding, we will give actual scenario descriptions during the process.

*1) In-Shard Credential Issuance/Verification:* We give a detailed scenario of in-shard drug purchases. When a patient CU visits a local hospital and purchases a particular prescription drug at a local pharmacy in the same shard through the medical DID-associated diagnosis and treatment record as credentials, an identity authentication request needs to be initiated to the pharmacy CU. In this scenario, the patient CU is the credential applier $\mathsf{CU}_A$, the hospital CU is the credential issuer $\mathsf{CU}_I$, and the pharmacy CU is the credential verifier $\mathsf{CU}_V$. $\mathsf{CU}_A$, $\mathsf{CU}_I$, and $\mathsf{CU}_V$ are all registered in $\mathsf{S}^1$.

The in-shard credential usage shown in Fig. 7 is performed in the following steps.

*Step 1:* $\mathsf{CU}_A$ sends a credential application request to $\mathsf{CU}_I$, including its DID and timestamp. The request should be signed by $\mathsf{CU}_A.usk$

*Step 2:* $\mathsf{CU}_I$ verifies the requests by using $\mathsf{CU}_A.upk$ on $\mathsf{S}^1$.C, generates the credential $\langle CRD\rangle_{\mathsf{CU}_I.usk}$ for $\mathsf{CU}_A$, and sends it to $\mathsf{CU}_A$.

*Step 3:* $\mathsf{CU}_I$ sends a $TX_{\mathrm{CR}} = \langle$'*CRD-Issue*', $DID_{\mathsf{CU}_A}$, $Hash(CRD)$, $DID_{\mathsf{CU}_I}\rangle_{\mathsf{CU}_I.usk}$ to $\mathsf{S}^1$.IMIs

*Step 4:* $\mathsf{S}^1$.IMIs run BFT to verify $TX_{\mathrm{CR}}$ and store the credential issue record on the C. The commitment proof is sent back to $\mathsf{CU}_I$.

After this step, the credential issuance record can be effectively traced by the blockchain.

*Step 5:* After receiving $\langle CRD\rangle_{\mathsf{CU}_I.usk}$ from $\mathsf{CU}_I$, the $\mathsf{CU}_A$ can use the credential for services. $\mathsf{CU}_A$ sends a credential usage request $RQ_{\mathrm{CRU}} = \langle DID_{\mathsf{CU}_A}, \langle CRD\rangle_{\mathsf{CU}_I.usk}, T\rangle_{\mathsf{CU}_A.usk}$ to $\mathsf{CU}_V$.

*Step 6:* After receiving $RQ_{\mathrm{CRU}}$ from $\mathsf{CU}_A$, $\mathsf{CU}_V$ verifies the request and sends $TX_{\mathrm{CV}} = \langle$'*CRD-Use*', $DID_{\mathsf{CU}_A}$, $Hash(CRD)$, $DID_{\mathsf{CU}_I}$, $DID_{\mathsf{CU}_V}\rangle_{\mathsf{CU}_V.usk}$ to $\mathsf{S}^1$.IMIs.

*Step 7:* $\mathsf{S}^1$.IMIs run BFT to verify $TX_{\mathrm{CV}}$, check the validity of the credential, and store the credential verification record on the C. The commitment proof is sent back to $\mathsf{CU}_V$.

After this step, the credential usage record can be effectively traced by the blockchain.

*2) Cross-Shard Credential Issuance/Verification:* The scenario is similar to in-shard credential usage. The difference is that the $\mathsf{CU}_A$, $\mathsf{CU}_I$, and $\mathsf{CU}_V$ are registered in different shards which correspond to different regions. For example, the patient travel to other cities and visit hospitals. After the trip, he/she needs to purchase the drug at the local pharmacy. $\mathsf{CU}_A$ and $\mathsf{CU}_V$ are registered in $\mathsf{S}^1$, $\mathsf{CU}_I$ is registered in $\mathsf{S}^2$.

The cross-shard credential usage shown in Fig. 7 is performed in the following steps.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

*Step 1:* $\mathsf{CU}_A$ sends a credential application request to $\mathsf{CU}_I$ including its DID and timestamp. The request should be signed by $\mathsf{CU}_A.usk$.

*Step 2:* $\mathsf{CU}_I$ verifies the requests, generates the credential $\langle CRD\rangle_{\mathsf{CU}_I.usk}$ for $\mathsf{CU}_A$, and sends it to $\mathsf{CU}_A$.

*Step 3:* $\mathsf{CU}_I$ sends a $TX_{\mathrm{CR}} = \langle$'*CRD-Issue*', $DID_{\mathsf{CU}_A}$, $Hash(CRD)$, $DID_{\mathsf{CU}_I}\rangle_{\mathsf{CU}_I.usk}$ to $\mathsf{S}^2$.IMIs.

*Step 4:* $\mathsf{S}^2$.IMIs run BFT to verify $TX_{\mathrm{CR}}$ and store the credential issue record on $\mathsf{S}^2$.C. And the commitment proof $\langle TX_{\mathrm{CR}}\rangle_{\sigma^2}$ is sent to $\mathsf{S}^1$.IMIs.

*Step 5:* After receiving $\langle TX_{\mathrm{CR}}\rangle_{\sigma^2}$ from $\mathsf{S}^2$.IMIs, $\mathsf{S}^1$.IMIs run BFT to check the proof and record it on $\mathsf{S}^1$.C.

*Step 6:* After receiving $\langle CRD\rangle_{\mathsf{CU}_I.usk}$ from $\mathsf{CU}_I$, the $\mathsf{CU}_A$ can use the credential for services. $\mathsf{CU}_A$ sends a credential usage request $RQ_{\mathrm{CRU}} = \langle DID_{\mathsf{CU}_A}, \langle CRD\rangle_{\mathsf{CU}_I.usk}, T\rangle_{\mathsf{CU}_A.usk}$ to $\mathsf{CU}_V$.

*Step 7:* After receiving $RQ_{\mathrm{CRU}}$ from $\mathsf{CU}_A$, $\mathsf{CU}_V$ verifies the request and sends $TX_{\mathrm{CV}} = \langle$'*CRD-Use*', $DID_{\mathsf{CU}_A}$, $Hash(CRD)$, $DID_{\mathsf{CU}_I}$, $DID_{\mathsf{CU}_V}\rangle_{\mathsf{CU}_V.usk}$ to $\mathsf{S}^1$.IMIs.

*Step 8:* $\mathsf{S}^1$.IMIs run BFT to verify $TX_{\mathrm{CV}}$, check the validity of the credential, and store the credential verification record on the C. The commitment proof is sent to $\mathsf{S}^2$.IMIs.

*Step 9:* After receiving commitment proof from $\mathsf{S}^1$.IMIs, $\mathsf{S}^2$.IMIs run BFT to check the proof and store the credential verification record on the C. The commitment proof is sent back to $\mathsf{CU}_V$. And the record is nonconflict between shards.

*3) Primitive Cross-Field Credential Verification:* In the cross-shard scenario, after a patient CU has received medical treatment and been discharged, they must pay for their medical expenses using their insurance account. This process requires patients to have control over their financial DID and medical DID beforehand. Insurance company CU must verify the patient's medical credentials to determine the appropriate compensation, which varies depending on the type of illness. Therefore, it is essential to make insurance companies recognize medical credentials through a cross-field process.

To illustrate the cross-field credential usage process, for simplicity, We assume that $\mathsf{CU}_A$ is registered in $\mathsf{F}^{f_1}.\mathsf{S}^1$ with $DID^{f_1}$ and $\mathsf{F}^{f_2}.\mathsf{S}^1$ with $DID^{f_2}$. $\mathsf{CU}_I$ is registered in $\mathsf{F}^{f_1}.\mathsf{S}^1$. $\mathsf{CU}_V$ is registered in $\mathsf{F}^{f_2}.\mathsf{S}^1$. The credential *CRD* issued by $\mathsf{CU}_I$ has been recorded on the $\mathsf{F}^{f_1}.\mathsf{S}^1$.C. And primitive cross-field credential verification shown in Fig. 8 is performed as following steps.

*Step 1:* After receiving $\langle CRD\rangle_{\mathsf{CU}_I.usk}$ from $\mathsf{CU}_I$, the $\mathsf{CU}_A$ can use the credential for services. $\mathsf{CU}_A$ sends a cross-field credential usage request $RQ_{\mathrm{CRU}} = \langle DID^{f_1}_{\mathsf{CU}_A}, DID^{f_2}_{\mathsf{CU}_A}, \langle CRD\rangle_{\mathsf{CU}_I.usk}, T\rangle_{\mathsf{CU}_A.usk_{f_2}}$ and $\langle RQ_{\mathrm{CRU}}\rangle_{\mathsf{CU}_A.usk_{f_1}}$ to $\mathsf{CU}_V$.

*Step 2:* After receiving $RQ_{\mathrm{CRU}}$ from $\mathsf{CU}_A$, $\mathsf{CU}_V$ verifies the request by $DID^{f_2}.upk$ and sends $TX_{\mathrm{CV}} = \langle$'*CRD-Use*', $DID^{f_1}_{\mathsf{CU}_A}$, $DID^{f_2}_{\mathsf{CU}_A}$, $Hash(CRD)$, $DID_{\mathsf{CU}_I}$, $DID_{\mathsf{CU}_V}\rangle_{\mathsf{CU}_V.usk}$ to field leader shard $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs.

*Step 3:* $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs run BFT to verify $TX_{\mathrm{CV}}$, check the validity of the *CRD*, generate an authorization commitment proof $\langle TX_{\mathrm{CV}}\rangle_{\sigma^{f_2.L}}$, and store the credential verification record on the C. The commitment proof is sent to $\mathsf{F}^{f_1}.\mathsf{S}^L$.IMIs.

*Step 4:* After receiving commitment proof from $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs, $\mathsf{F}^{f_1}.\mathsf{S}^L$.IMIs run BFT to check the authorization proof and send new authorization proof $\langle TX_{\mathrm{CV}}\rangle_{\sigma^{f_1.L}}$ to $\mathsf{F}^{f_1}.\mathsf{S}^1$.IMIs.

*Step 5:* After receiving authorization proof from $\mathsf{F}^{f_1}.\mathsf{S}^L$.IMIs, $\mathsf{F}^{f_1}.\mathsf{S}^1$.IMIs run BFT to verify the authorization proof, check the validity of credential, and send the commitment proof including $\mathsf{CU}_A.upk_{f_1}$ signed by $\sigma^{f_1.1}$ to $\mathsf{F}^{f_1}.\mathsf{S}^L$.IMIs. The usage of *CRD* will be recorded on $\mathsf{F}^{f_1}.\mathsf{S}^1$.IMI.C.

*Step 6:* After receiving commitment proof from $\mathsf{F}^{f_1}.\mathsf{S}^1$.IMIs, $\mathsf{F}^{f_1}.\mathsf{S}^L$.IMIs run BFT to check the proof, generate new commitment proof signed by $\sigma^{f_1.L}$, and send new proof to $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs.

*Step 7:* After receiving commitment proof from $\mathsf{F}^{f_1}.\mathsf{S}^1$.IMIs, $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs run BFT to check the proof, generate new commitment proof signed by $\sigma^{f_2.L}$, and send new proof to $\mathsf{F}^{f_2}.\mathsf{S}^1$.IMIs.

*Step 8:* After receiving commitment proof from $\mathsf{F}^{f_2}.\mathsf{S}^L$.IMIs, $\mathsf{F}^{f_2}.\mathsf{S}^1$.IMIs run BFT to check the validity and verify the $\langle RQ_{\mathrm{CRU}}\rangle_{\mathsf{CU}_A.usk_{f_1}}$ by $\mathsf{CU}_A.upk_{f_1}$ to make sure the binding of $DID^{f_1}$ and $DID^{f_2}$.

*4) Cross-Field Optimization:* During the cross-field usage, the online authorization and verification process operated like Section IV-C3 is less efficient. So, we introduce the Merkle proof to optimize the process by following steps. The time is divided into epochs. The optimization of cross-field credential usage is shown in Fig. 8.

*Step 1:* At the beginning of each epoch $e_k$, each shard $\mathsf{F}^{f_j}.\mathsf{S}^i$.IMIs run BFT to generate new a credential record list $CRL_{k-1}$ in $e_{k-1}$ as Merkle tree $MT^{f_j.i}_{k-1}$. The Merkle root is sent to leader shard $\mathsf{F}^{f_j}.\mathsf{S}^L$.IMIs.

*Step 2:* After receiving the Merkle root of the message, the field leader shard $\mathsf{F}^{f_j}.\mathsf{S}^L$.IMIs compose a whole field Merkle Tree $MT^{f_j}_{k-1}$.

*Step 3:* Before the credential verification, $\mathsf{CU}_A$ applies a Merkle proof for credential query $M^C_p$ from $\mathsf{F}^{f_j}.\mathsf{S}^i$.IMIs and a Merkle proof for cross-field authorization $M^A_p$ from $\mathsf{F}^{f_j}.\mathsf{S}^L$.IMIs. The generation of Merkle proof is shown in Algorithm 1

*Step 4:* While cross-field verification occurs, the $\mathsf{CU}_V$ only needs to verify the credentials from signature and Merkle proof. $M^C_p$ and $M^A_p$ through $\mathsf{F}^{f'_j}.\mathsf{S}^i$ and $\mathsf{F}^{f'_j}.\mathsf{S}^L$. The verification of Merkle proof is shown in Algorithm 2

Furthermore, after many system epochs, different fields can generate a new Merkle tree and send it to the main chain MC. Through some smart contract design, the verification will be further optimized and achieve extreme efficiency.

To be noticed, this optimization is especially suitable for long-term, nonsensitive to usage times credentials (the corresponding entity information and attributes are rarely modified after being created, and verification operations are frequent), such as the factory information of IoT devices, basic

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

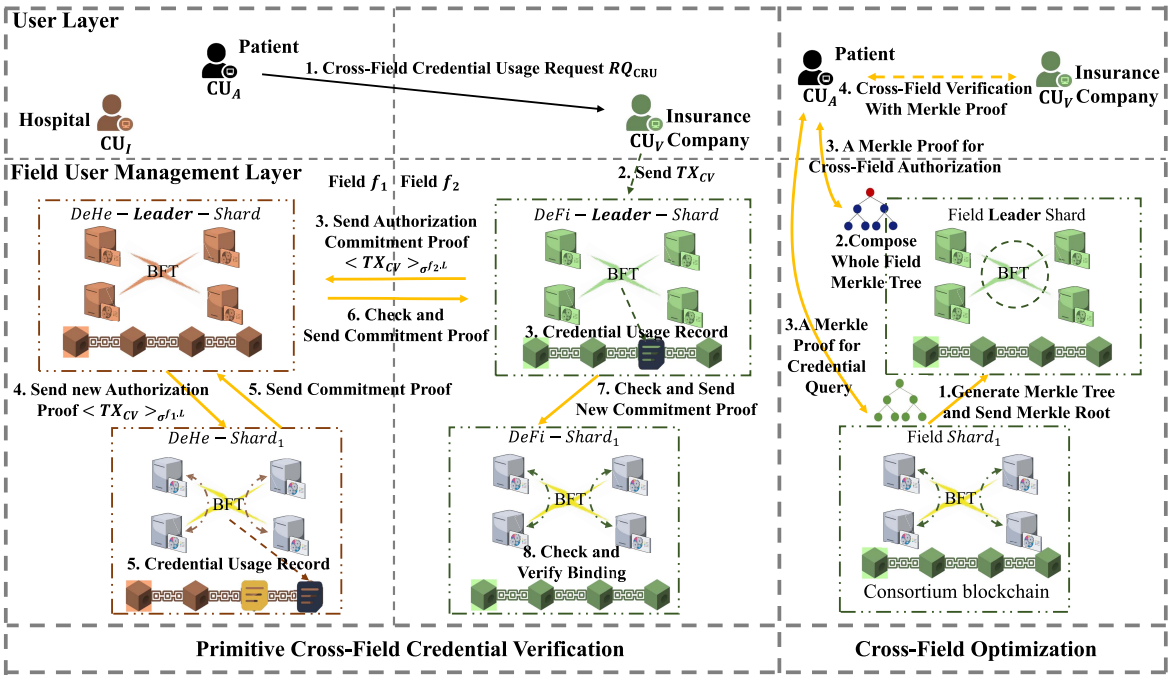



Fig. 8. Cross-field usage of credential.

personal identity information, personal physical examination information, etc.

*5) Optional Credential Authorization Usage:* In more realistic scenarios, users have more complex credential authorization usage, such as children helping older people apply for medical credentials, salespeople in insurance companies using manager-level DID private keys to issue policy credentials, etc. For simplicity, in this process, we explain the process of credential delegation from two levels and give a brief system description.

1) *DID Authorization Level:* In this level, the user $\mathsf{CU}_0$ operates the similar process in Section IV-B4 to finish DID authorization. When a delegated user $\mathsf{CU}_1$ constructs a credential-related request, $\mathsf{CU}_1$ needs to indicate the $\mathsf{CU}_0$ ownership of the request. All requests will recorded on the chain. During credential usage, the DID authorization record in the DID document needs to be verified simultaneously during the credential verification.
2) *Credential Authorization Level:* In this level, the user $\mathsf{CU}_0$ only authorizes some credentials' usage to $\mathsf{CU}_1$ but not the usage rights of the DID private key *usk* to $\mathsf{CU}_1$. $\mathsf{CU}_0$ only attaches the delegation attributes of the credentials when applying, including the delegated user $\mathsf{CU}_1$ and the authorization level. All these records need to be assured by $\mathsf{S}^i$.IMIs through BFT to ensure effective management of the credentials.

### D. IMI Shard Management

DID management in Web3 is dynamic to a certain extent. IMI shards may change as the real organization changes. New field user management shards may need to be registered or deleted, similar to possible adjustments of the certificate authority in traditional Internet. In addition, considering fairness, field leader shards and maintenance of the main chain also need to be rotated regularly. Therefore, in this part, we design a sharding management solution that is easy to understand and relatively open regarding functions and implementation details as Fig. 9 depicts. The shard management includes different levels. In this section, we mainly focus on the management of the field user management layer.

*Step 1:* When an $\mathsf{IMI}_{\text{new}}$ of $\mathsf{F}^1$ enters the SS-DID system, it generates a key pair $(apk, ask)$ and sends an authentication request to $\mathsf{F}^1.\mathsf{S}^L.$IMIs.

*Step 2:* After receiving an authentication request from $\mathsf{IMI}_{\text{new}}$, $\mathsf{F}^1.\mathsf{S}^L.$IMIs run BFT to verify the request, allocate a shard $\mathsf{S}^i$ for $\mathsf{IMI}_{\text{new}}$, record the node message, generate the commitment proof $\langle$*'New-IMI'*, $apk$, $\mathsf{IMI}_{\text{new}}\rangle_{\sigma^L}$, and send it to $\mathsf{S}^i.$IMIs.

*Step 3:* After receiving $\langle$*'New-IMI'*, $apk$, $\mathsf{IMI}_{\text{new}}\rangle_{\sigma^L}$ from $\mathsf{S}^L.$IMIs, $\mathsf{S}^i.$IMIs run BFT to verify the proof and let $\mathsf{IMI}_{\text{new}}$ join into $\mathsf{S}^i$. For the smooth operation of the system, when the new epoch $e_k$ starts, the $\mathsf{IMI}_{\text{new}}$ participates in the system operation.

*Step 4:* The shard exchange record will be uploaded as a Merkle tree to the main chain MC by $\mathsf{S}^L$.

The node exit process is similar to node entry. The slight difference is that the public key of the exit node should be kept for old credential verification.

## V. Security Analysis

In this section, we prove our scheme meets all security goals mentioned in Section III-C.

*Definition 1 (Unforgeability of DID):* $\forall\ \mathcal{A}$ cannot forge $\langle$DID-related Message$\rangle_{\sigma^i}$ s.t. CU regards $\sigma^i$ as a valid BFT

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

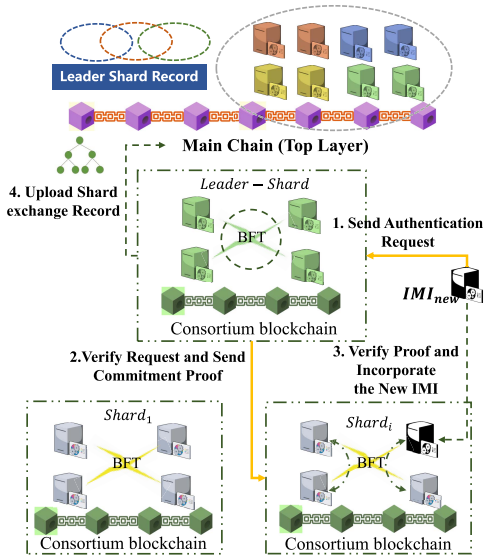



Fig. 9. IMI shard management.

commitment certificate from $S^i$.IMIs and regards messages as valid.

*Theorem 1 (SS-DID Satisfies Unforgeability of DID):* Assume that $\mathcal{A}$ is restricted by system assumptions, then SS-ADS-B satisfies unforgeability of DID as per Definition 1.

*Proof of Theorem 1:* Under the adversary and communication model, according to BFT algorithm, when $\mathcal{A}$ impersonates up to 1/3 IMI nodes to send incorrect messages in single shard $S^i$ process, the honest nodes can achieve consistency of DID message and generate the signature proofs $\langle$DID-related Message$\rangle_{\sigma^i}$. In the cross-shard process, $\mathcal{A}$ cannot forge the proof of each shard, so $\mathcal{A}$ cannot create conflict between different shards. During the cross-field process, $\mathcal{A}$ cannot forge the proof of leader shard $S^L$, so $\mathcal{A}$ cannot make nontrust between different field shard$F^j$.$S^i$s. So DID-related message, which IMI nodes generate, can achieve unforgeability. ■

*Definition 2 (System Liveness):* A submitted request must be processed after a finite-time period $t$.

*Theorem 2 (SS-DID Satisfies System Liveness):* Assume that $\mathcal{A}$ is restricted by system assumptions, then SS-DID satisfies system liveness as per Definition 2.

*Proof of Theorem 2:* We prove the system liveness property and calculate the theoretical operation time of all kinds of system behaviors in Table II.

In conclusion, under the adversary and communication model, the liveness property of BFT can be held while other steps can end in a specific time, so the system must finish all procession after a finite time $t$. ■

## VI. Performance Evaluation

In this section, we evaluate SS-DID through simulation and illustrate that our scheme meets all scalability and performance goals mentioned in Section III-C.

### A. Simulation Set

In the simulation, we set the communication bandwidth to 100 Mb/s and communication delay to 50 ms between CUs and IMIs. We assume that each request package is 1 kB and the batch size of the request process is 2000. That is to say, all BFT algorithms propose blocks as 2 MB. The IMIs of each shard vary from 30 to 60. Especially in the TPS simulation of the cross-field scenario, the in-field IMIs are fixed to 600, and total shards vary from 50 to 100 in 5 fields. We use SHA-256 as the hash function to construct the Merkle tree. The Merkle proof We choose baseline BFT algorithms, including PBFT, simple Hotstuff, and pipelined-Hotstuff.

TABLE II
System Operation Time

| System behaviour | System operation time |
|---|---|
| DID Registration | $2*\delta_{CU}+T_{BFT}=O(\delta_{CU}+T_{BFT})$ |
| DID Update | $2*\delta_{CU}+T_{BFT}=O(\delta_{CU}+T_{BFT})$ |
| DID Delete | $2*\delta_{CU}+T_{BFT}=O(\delta_{CU}+T_{BFT})$ |
| DID Authorization | Nearly the same as DID Update |
| In-Shard CRD-Iss | $4*\delta_{CU}+T_{BFT}=O(\delta_{CU}+T_{BFT})$ |
| In-Shard CRD-Ver | $4*\delta_{CU}+T_{BFT}=O(\delta_{CU}+T_{BFT})$ |
| Cross-Shard CRD-Iss | $4*\delta_{CU}+\delta_X+2*T_{BFT}=O(\delta_{CU}+\delta_X+T_{BFT})$ |
| Cross-Shard CRD-Ver | $4*\delta_{CU}+\delta_X+2*T_{BFT}=O(\delta_{CU}+\delta_X+T_{BFT})$ |
| Cross-Field CRD-Ver(P) | $4*\delta_{CU}+6*\delta_X+6*T_{BFT}=O(\delta_{CU}+\delta_X+T_{BFT})$ |
| Cross-Field CRD-Ver(OP) | $O(\delta_{CU}+\delta_X+\delta_{MPV}+T_{BFT})$ |

The symbol "$\delta_{CU}$" = the time of CU of communication with each other or S.IMIs, "$\delta_X$" = the time of cross shard communication, "$\delta_{MPV}$" = the time of Merkle proof verification.

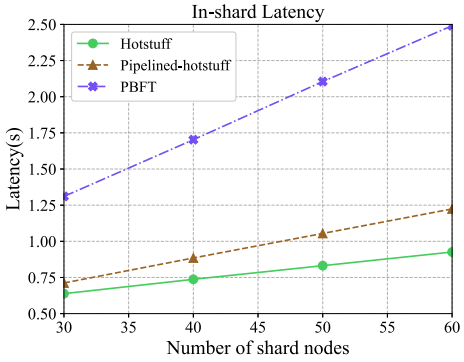



Fig. 10. In-shard scenario latency under different IMI node numbers.

### B. Latency

We simulate and evaluate DID life cycle management authentication and the most complex cross-field credential verification latency of the SS-DID system under different BFT algorithms. And Figs. 10 and 11 show the results.

The latency increases as node number rises and varies based on the consensus algorithm used. In the *best case*, the in-shard latency is less than 0.7 s, and the cross-field latency is less than 2 s.

### C. Throughput

We simulate and evaluate the overall process throughput of the proposed SS-DID system under different BFT algorithms. Fig. 12 shows the overall system throughput. The throughput refers to how many requests can be handled in a second.

From Fig. 12, the total throughput gradually increases with the growth of shards. The result illustrates that our system can achieve scalability through sharding technology. For five fields, 20 shards per field, and 30 nodes per shard, the best

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

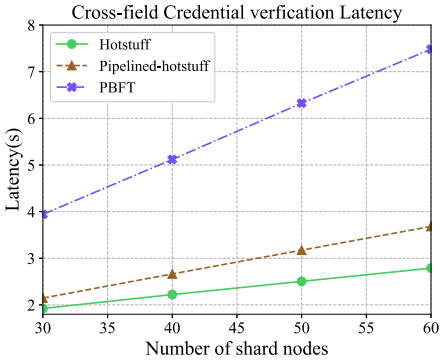



Fig. 11. Cross-field credential verification latency under different IMI node numbers.

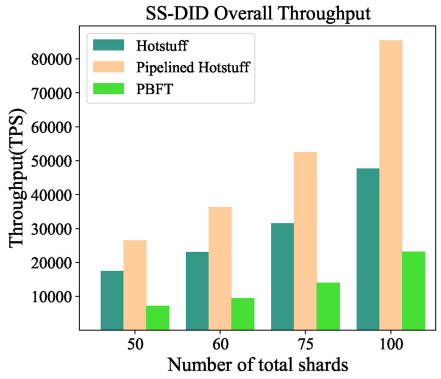



Fig. 12. SS-DID System overall throughput under different nodes and shards.

overall system throughput can reach 90 KTPS. Furthermore, the optimized BFT algorithm can be used as a pluggable component to improve the system throughput much more.

## VII. Discussion and Conclusion

In this article, we propose SS-DID, a secure and scalable Web3 DID system utilizing multilayer sharding blockchain. Our solution divides identity management nodes according to different fields to achieve compatibility. In-field node sharding improves the scalability of DID-related transaction processing, while leader shard achieves cross-field mutual trust and management by maintaining the main chain. The results of the analysis and evaluation prove the effectiveness of the scheme.

In conclusion, the proposed system is expected to revolutionize the whole Web3 by providing a secure and scalable DID solution. Further, future work may include the usage of new distributed ledger based on directed acyclic graph (DAG) to achieve higher efficient DID updates and queries, original distributed threshold cryptography design to realize anonymous but threshold-auditable credentials, and some user-centered DID self-management methods.

## References

[1] "What is web3?" 2024. [Online]. Available: https://ethereum.org/web3
[2] T. Berners-Lee, "Semantic Web road map," 1998. [Online]. Available: https://www.w3.org/DesignIssues/Semantic.html
[3] W. Gavin, "Dapps: What Web 3.0 looks like." 2017. [Online]. Available: https://gavwood.com/dappsweb3.html
[4] C. Chi et al., "White paper-global Web3 technology industry ecological development report," Inst. Elect. Electron. Eng., Piscataway, NY, USA, White Paper, 2023.
[5] C. R. Harvey, A. Ramachandran, and J. Santoro, *DeFi and the Future of Finance*. Hoboken, NJ, USA: Wiley, 2021.
[6] W. Ding et al., "DeSci based on Web3 and DAO: A comprehensive overview and reference model," *IEEE Trans. Comput. Soc. Syst.*, vol. 9, no. 5, pp. 1563–1573, Oct. 2022.
[7] S. Wang, W. Ding, J. Li, Y. Yuan, L. Ouyang, and F. Wang, "Decentralized autonomous organizations: Concept, model, and applications," *IEEE Trans. Comput. Soc. Syst.*, vol. 6, no. 5, pp. 870–878, Oct. 2019.
[8] M. Alizadeh, K. Andersson, and O. Schelén, "Comparative analysis of decentralized identity approaches," *IEEE Access*, vol. 10, pp. 92273–92283, 2022.
[9] P. P. Ray, "Web3: A comprehensive review on background, technologies, applications, zero-trust architectures, challenges and future directions," *Internet Things Cyber Phys. Syst.*, vol. 3, pp. 213–248, May 2023.
[10] O. Dib and K. Toumi, "Decentralized identity systems: Architecture, challenges, solutions and future directions," *Ann. Emerg. Technol. Comput.*, vol. 4, no. 5, pp. 19–40, 2020.
[11] O. Avellaneda et al., "Decentralized identity: Where did it come from and where is it going?" *IEEE Commun. Stand. Mag.*, vol. 3, no. 4, pp. 10–13, Dec. 2019.
[12] E. Samir, H. Wu, M. Azab, C. Xin, and Q. Zhang, "DT-SSIM: A decentralized trustworthy self-sovereign identity management framework," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 7972–7988, Jun. 2022.
[13] E. S. Fathalla, M. Azab, C. Xin, and H. Wu, "PT-SSIM: A proactive, trustworthy self-sovereign identity management system," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17155–17169, Oct. 2023.
[14] B. C. Ghosh et al., "Decentralized cross-network identity management for blockchain interoperation," in *Proc. IEEE ICBC*, 2021, pp. 1–9.
[15] (Decentralized Identity Found., Los Gatos, CA, USA). *Together We're Building a New Identity Ecosystem*. 2022, Accessed: Jan. 23, 2024. [Online]. Available: https://identity.foundation
[16] (W3C, Cambridge, MA, USA). *Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations*. 2022. Accessed: Jan. 23, 2024. [Online]. Available: https://www.w3.org/TR/did-core
[17] D. Maram et al., "CanDID: Can-do decentralized identity with legacy compatibility, Sybil-resistance, and accountability," in *Proc. IEEE SP*, 2021, pp. 1348–1366.
[18] L. Xia et al., "DIDAPPER: A practical and auditable on-chain identity service for decentralized applications," in *Proc. IEEE DAPPS*, 2023, pp. 151–157.
[19] X. Fan, Q. Chai, L. Xu, and D. Guo, "DIAM-IoT: A decentralized identity and access management framework for Internet of Things," in *Proc. BSCI*, 2020, pp. 186–191.
[20] R. Xiong, W. Ren, X. Hao, J. He, and K. R. Choo, "BDIM: A blockchain-based decentralized identity management scheme for large scale Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22581–22590, Dec. 2023.
[21] "Decentralized-identity." Microsoft. 2024, Accessed: Jan. 23, 2024. [Online]. Available: https://www.microsoft.com/en-us/security/business/solutions/decentralized-identity
[22] "Hyperledger Indy." Hyperledger. 2024. Accessed: Jan. 23, 2024. [Online]. Available: https://www.hyperledger.org/projects/hyperledger-indy
[23] "Home-sovrin." Sovrin. 2024. Accessed: Jan. 23, 2024. [Online]. Available: https://sovrin.org
[24] N. Naik and P. Jenkins, "uPort open-source identity management system: An assessment of self-sovereign identity and user-centric data platform built on blockchain," in *Proc. IEEE, ISSE*, 2020, pp. 1–7.
[25] S. Huh, M. Shim, J. Lee, S. S. Woo, H. Kim, and H. Lee, "DID we miss anything?: Towards privacy-preserving decentralized ID architecture," *IEEE Trans. Depend. Secur. Comput.*, vol. 20, no. 6, pp. 4881–4898, Dec. 2023.
[26] K. Wang et al., "Hades: Practical decentralized identity with full accountability and fine-grained Sybil-resistance," in *Proc. ACSAC*, 2023, pp. 216–228.
[27] F. Kirstein, A. Altenbernd, S. Schimmler, and M. Hauswirth, "A decentralised persistent identification layer for DCAT datasets," in *Proc. WWW Compan.*, 2023, pp. 1424–1427.
[28] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system." 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf
[29] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OSDI*, 1999, pp. 173–186.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.

[30] A. N. Bessani, J. Sousa, and E. A. P. Alchieri, "State machine replication for the masses with BFT-SMART," in *Proc. DSN*, 2014, pp. 355–362.
[31] Y. Amoussou-Guenou, A. D. Pozzo, M. Potop-Butucaru, and S. Tucci Piergiovanni, "Correctness and fairness of tendermint-core blockchains," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2018/574, 2018.
[32] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *Proc. PODC*, 2019, pp. 347–356.
[33] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. CCS*, 2016, pp. 17–30.
[34] Y. Liu et al., "Secure and scalable cross-domain data sharing in zero-trust cloud-edge-end environment based on sharding blockchain," *IEEE Trans. Depend. Secur. Comput.*, early access, Sep. 11, 2023, doi: 10.1109/TDSC.2023.3313799.
[35] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE SP*, 2018, pp. 583–598.
[36] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. CCS*, 2018, pp. 931–948.
[37] Y. Liu, J. Liu, Q. Wu, H. Yu, Y. Hei, and Z. Zhou, "SSHC: A secure and scalable hybrid consensus protocol for sharding blockchains with a formal security framework," *IEEE Trans. Depend. Secur. Comput.*, vol. 19, no. 3, pp. 2070–2088, Jun. 2022.
[38] Y. Liu et al., "A flexible sharding blockchain protocol based on cross-shard Byzantine fault tolerance," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2276–2291, 2023.
[39] A. Liu et al., "CHERUBIM: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3178–3193, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10415090
[40] R. C. Merkle, "One way hash functions and DES," in *Proc. CRYPTO*, 1989, pp. 428–446.



**Yizhong Liu** (Member, IEEE) received the B.S. degree in electronic and information engineering and the Ph.D. degree from Beihang University, Beijing, China, in 2014 and 2021, respectively.

He is currently an Assistant Professor with the School of Cyber Science and Technology, Beihang University. He has authored over 30 publications, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING. His research interests include cryptography and blockchain.



**Boyu Zhao** received the B.S. degree in information security from Beihang University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology.

His research interests include cryptography and blockchain consensus.



**Zedan Zhao** will receive the B.S. degree in information security from Beihang University, Beijing, China, in 2024, where she is going to pursue the master's degree with the School of Cyber Science and Technology.

Her research interests include privacy protection, cryptography, and information security.



**Jianwei Liu** (Senior Member, IEEE) received the B.S. and M.S. degrees in electronic and information from Shandong University, Jinan, China, in 1985 and 1988, respectively, and the Ph.D. degree in communication and electronic system from Xidian University, Xi'an, China, in 1998.

He is currently a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. His current research interests include wireless communication networks, cryptography, and information security.



**Xun Lin** is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, Beihang University, Beijing, China.

His research interests include image manipulation detection and medical image analysis.



**Qianhong Wu** (Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2004.

He is an Associate Research Fellow with the University of Wollongong, Wollongong, NSW, Australia, and a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. He has authored over 200 publications and served on the program committees of several international conferences on information security and privacy. His research interests include cryptography, and information security and privacy.



**Willy Susilo** (Fellow, IEEE) received the master's and Doctor of Philosophy degrees from the University of Wollongong, Wollongong, NSW, Australia, in 1996 and 2001, respectively.

He is a Professor and the Head of the School of Computing and Information Technology, University of Wollongong, where he is also the Director of the Centre for Computer and Information Security Research. His main research interests include cloud security, cryptography, and information security.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:17:03 UTC from IEEE Xplore. Restrictions apply.